

本组架构方案调研与选型决策

一、 本组项目场景

本组选择在线书店电商场景，专注于纸质书的零售交易。主要交易对象为实体书。主要服务对象为普通读者、学生及藏书爱好者。项目准备完成的核心功能范围含注册登录、密码加密传输、商品详情页、购物车、订单支付、支付信息加密、个人中心、商品管理、订单处理

二、 方案调研

单体架构：

定义：所有功能模块（用户、商品、订单、支付等）耦合在单一代码库中，打包为单个部署包运行

成本：单一代码库、单一部署单元，无需服务发现、RPC、分布式事务等基础设施，开发上手快。

项目周期：构建、测试、部署链路短，适合快速上线验证，后续迭代在早期也较快。

团队匹配度：小团队或全栈团队协作成本低，统一技术栈，不需要同时维护多个服务。

安全边界：模块间缺乏强制隔离和网络边界，权限控制容易混在业务代码中，一个模块漏洞可能影响整个系统。

可维护性：随着商品、订单、库存等模块增长，代码耦合度上升，修改一处容易影响其他模块，维护成本明显增加。

扩展能力：只能整体水平扩展，无法对订单、搜索等热点模块单独扩容；数据库容易成为全局瓶颈。

前后端分离架构：

成本：介于单体与服务化之间，需搭两套工程与联调环境，略高于单体。

项目周期：前后端解耦后可并行开发、并行调试，任务可分给不同组员同步推进，利于在项目规定时间内完成；需预留少量接口联调时间

团队匹配度：本组具备 Java Web 基础，Vue 上手曲线平缓、教程与官方文档充分；按"两人前端、其余后端"分工与现有基础较匹配。

安全边界：认证/鉴权/数据访问全部收敛在后端API，边界清晰，前端不直连数据库。

可维护性：前后端代码分离、模块职责单一，页面问题与接口问题可独立定位。做好接口处理以及项目文档便于小组协作与修改。

扩展能力：同一套后端API可同时服务PC端、后续小程序/移动端；后端接口稳定后可按业务拆分为服务化架构

服务化架构：

成本：成本最高，需要同时具备服务开发+DevOps开发+API设计，同时有一定的服务器需求，在规模较大时成本可接受。

项目周期：服务化架构的项目周期高度取决于迁移策略。全量重写风险最高、周期最长；绞杀者模式则允许渐进式迁移。但两种策略所需时间都以月为单位，不适宜小规模开发。

团队匹配度：服务化架构对团队的要求远高于单体和前后端分离，需要的人数更多，同时需要协调各个服务间的代码，考验管理能力，不适配3~5人的小团队。

安全边界：传统单体应用的安全模型依赖集中式网关和会话管理，通常只需保护一个网络边界和一个数据库。微服务架构下，每个服务成为独立的安全域，服务间通信通过网络进行，存在大量暴露的端口和交互点。一旦某个服务被攻破，攻击者能够获取的权限和数据量可以被限制在单个服务范围内。

可维护性：服务化架构带来了很高的可维护性。单个单个服务故障不会影响整个系统，通过熔断、降级、限流等容错机制可以避免单点故障扩散到全链路。每个服务可以独立开发、部署和扩展，某个微服务发生变更时无需编译、部署整个应用，各服务

独立部署。

扩展能力：服务化架构的扩展能力是其相对于单体架构最具经济价值的优势，其能精准按需扩展而非“全家桶”式扩展。扩展能力的本质在于服务边界隔离：每个服务根据自身负载特征独立扩容，高并发服务（如秒杀场景的订单服务）可获得更多资源，而低频服务（如用户管理）保持最小实例数即可。

三、 多维评价

评价维度	权重	单体架构	前后端分离	服务化架构	评分依据
开发成本	15%	4	4	3	5人团队的代码开发量有限，学生项目不宜成本太高
项目周期	20%	4	4	2	除服务化以外应当能在规定时间内完成
团队匹配度	20%	4	4	2	成员大多第一次参与项目。
安全边界	15%	2	4	5	身份认证、权限控制和数据访问边界是否清晰
可维护性	15%	2	4	5	前后端与服务化架构的模块分离都比较易于维护。
扩展能力	15%	1	5	5	单体架构扩展能力较弱。
综合得分	100%	2.95	4.15	3.5	

四、参考资料

1. Spring Boot官方文档，该资料说明Spring Boot内嵌容器、自动配置、单一可运行jar/war的特点，使单体架构在开发、部署和团队协作上成本较低。
2. Martin Fowler《MonolithFirst》行业文章，该资料说明单体架构在业务规模增长后维护难度上升、扩展能力受限。
3. CSDN《前后端分离架构概述》该资料说明前后端分离架构下前端与后端相互独立、可并行开发和独立部署，因此能够支持本组多人并行开发、缩短项目周期、前端页面与后端接口独立维护的判断。
4. CSDN《Java Web架构实战篇：聊一聊前后端分离架构》该资料说明前后端分离能够解决小团队"一人前后端兼顾、互相阻塞"的效率问题，因此本组可以将前后端任务分派不同组员同步开发，易于团队协作。
5. 开源项目newbee-mall，该项目同时维护单体版本和前后端分离版本，newbee-mall的前后端分离版本将前端代码完全独立为Vue 3项目，前端团队可以使用Vue生态的完整工具链，后端仅暴露RESTful API接口，前端通过Axios调用。证明前后端分离架构确实可以分开管理，便于维护。

五、选型结论

综合网上书店、票务及综合商城场景需求，本项目最终选定前后端分离架构。该架构开发成本低于服务化方案，契合当前团队规模；相比单体架构，具备更好的可维护性与多端扩展能力。其有效平衡了开发效率与系统解耦需求，满足当前业务快速迭代要求，为后续架构平滑演进奠定基础。